

**Evaluación y comparación de plugins CNI para
Kubernetes:
un enfoque orientado a QoS, seguridad y eficiencia de
recursos en entornos de microservicios**

**Duwan Estiven Sierra Guerrero
Holman Audrey Alba Castro**

Director Gerardo Alberto Castang Montiel
Codirector

Ingeniería Telemática
Telemática - Grupo de Investigación ORION
Universidad Distrital Francisco José de Caldas
Bogotá, D.C. - Colombia
22 de mayo de 2026

Agradecimientos

GRACIAS

Resumen

Kubernetes no impone un plugin de red específico, lo que obliga a los equipos a seleccionar un *Container Network Interface* (CNI) sin métricas comparativas objetivas. Esta situación conduce a sobredimensionamiento de infraestructura (CPU/RAM) y a posturas de seguridad incompletas. El presente trabajo construye evidencia empírica reproducible —infraestructura como código, benchmarks automatizados y modelo de decisión multi-criterio (MCDA)— para guiar esa selección. Se evalúan Flannel, Calico, Cilium y Antrea en un clúster K3s en alta disponibilidad sobre DigitalOcean, midiendo *throughput* TCP, latencia de establecimiento, retransmisiones y overhead de CPU/RAM del agente CNI, así como el impacto de políticas de red Zero Trust. Los resultados alimentan un modelo MCDA parametrizable por perfil de uso (URLLC, Edge/IoT, microservicios transaccionales) y un prototipo web que expone las recomendaciones.

Palabras clave: Kubernetes, CNI, QoS, iperf3, Calico, Cilium, Flannel, Antrea, NetworkPolicy, Zero Trust, MCDA, K3s, DigitalOcean, GitOps, eBPF.

Abstract

Kubernetes does not impose a specific network plugin, forcing teams to select a *Container Network Interface* (CNI) without objective comparative metrics. This situation leads to infrastructure oversizing (CPU/RAM) and incomplete security postures. This paper builds reproducible empirical evidence —infrastructure as code, automated benchmarks, and a multi-criteria decision analysis (MCDA) model— to guide this selection. Flannel, Calico, Cilium, and Antrea are evaluated on a high-availability K3s cluster on DigitalOcean, measuring TCP throughput, establishment latency, retransmissions, and CPU/RAM overhead of the CNI agent, as well as the impact of Zero Trust network policies. The results feed a parameterizable MCDA model by use profile (URLLC, Edge/IoT, transactional microservices) and a web prototype that displays the recommendations.

Keywords: Kubernetes, CNI, QoS, iperf3, Calico, Cilium, Flannel, Antrea, Network-Policy, Zero Trust, MCDA, K3s, DigitalOcean, GitOps, eBPF.

Tabla de contenidos

Resumen	II
Abstract	III
Tabla de Contenido	VII
Lista de tablas	IX
Lista de figuras	X
1. Introducción	1
2. Estado del Arte	2
3. Planteamiento del Problema	3
4. Justificación	4
5. Objetivos	5
6. Marco Teórico - Conceptual	6
1. El modelo de red de Kubernetes	6
1.1. Identidad de red del Pod	6
1.2. Nodos y reenvío de paquetes	6
1.3. Direccionamiento estable: Services y EndpointSlices	7
1.4. Ingreso de tráfico externo	7
1.5. Aislamiento lógico: Namespaces y NetworkPolicies	7
1.6. Recorrido de una solicitud	7
2. Container Network Interface (CNI)	8
2.1. Especificación y funciones núcleo	8
2.2. Modelos de dataplane: overlay y ruteo nativo	8
2.3. Gestión de direcciones (IPAM)	9
2.4. MTU y fragmentación en redes con encapsulación	9
2.5. Observabilidad del plano de datos	9
2.6. Métricas de rendimiento y sobredimensionamiento	10
7. Metodología y Diseño de la Solución	11
1. Diseño de la Solución	11
1.1. Arquitectura de Red y Topología	11
1.1.1. Red underlay	11
1.1.2. Red overlay y CNI evaluados	12
1.2. Sistema de Medición de Calidad de Servicio (QoS)	13

1.2.1.	Inyección de tráfico	13
1.2.2.	Variables telemáticas capturadas	13
1.2.3.	Pila de observabilidad: Prometheus y Grafana	14
1.3.	Modelo de Decisión Multicriterio (MCDA)	14
1.3.1.	Limpieza estadística y normalización	14
1.3.2.	Matriz de pesos por perfil de usuario	15
1.4.	Seguridad y Micro-segmentación (Zero Trust)	15
1.4.1.	Estrategia de Default Deny	16
1.4.2.	Automatización GitOps con ArgoCD	16
1.5.	Flujo Integral del Sistema	16
1.6.	Arquitectura por Capas	17
1.7.	Trazabilidad y Requisitos de Diseño	17
1.8.	Prototipo Recomendador	18
1.9.	Alcance y Límites del Diseño	18
2.	Implementación de la Solución	18
2.1.	Aprovisionamiento de Infraestructura con Terraform	19
2.1.1.	Estructura del Código Terraform (K8s-bootstrap/00.bootstrap)	19
2.2.	Configuración del Clúster K3s en Alta Disponibilidad	19
2.2.1.	Base de Datos Externa para el Plano de Control	19
2.2.2.	Instalación: clúster sin CNI para plugins externos y modo nativo para Flannel	20
2.3.	Orquestación y Entrega Continua: GitOps con ArgoCD	20
2.3.1.	El Patrón App of Apps	21
2.3.2.	Configuración de selfHeal y prune	21
2.3.3.	Overlays de Kustomize para los CNIs	21
2.4.	Implementación del Stack de Observabilidad	22
2.4.1.	Prometheus: El Recolector de Métricas	22
2.4.2.	Grafana: Los Dashboards de Red	22
2.4.3.	Resource Usage Exporter: Extracción Automatizada de Datos	22
2.5.	Implementación del Banco de Pruebas de QoS	23
2.5.1.	Escenario de Rendimiento con iperf3	23
2.5.2.	Cliente de Latencia Personalizado	23
2.6.	Procesamiento Estadístico y Aplicación del Modelo MCDA	24
2.6.1.	Limpieza con el Rango Intercuartílico (IQR)	24
2.6.2.	Normalización y Cálculo del Score MCDA	24
2.7.	Prototipo Recomendador: Aplicación Web SPA	25
2.7.1.	Tecnologías de Implementación	25
2.7.2.	Flujo de la Aplicación	25
2.8.	Implementación de Network Policies y Seguridad Zero Trust	25
2.8.1.	Políticas Default Deny	25
2.8.2.	Micro-segmentación por Capas de Aplicación	26

2.8.3.	Validación Automatizada de Políticas	26
2.9.	Estructura de Repositorios y Entregables Técnicos	26
2.10.	Desafíos de Implementación y Soluciones Adoptadas	26
2.11.	Flujo completo de ejecución	28
8.	Resultados y Pruebas	29
1.	Metodología General de Pruebas	29
1.1.	Control de Condiciones: el Escenario de Referencia (Baseline) . . .	29
1.2.	Automatización: de la Prueba Manual a la Prueba Orquestada . . .	30
2.	Pruebas de Rendimiento de Red — Objetivo Específico 1	30
2.1.	Protocolo de Prueba de Throughput con iperf3	30
2.2.	Resultados de Throughput TCP por CNI	31
2.3.	Protocolo de Prueba de Latencia TCP	31
2.4.	Resultados de Latencia TCP por CNI	31
3.	Pruebas de Seguridad con Network Policies — Objetivo Específico 2	32
3.1.	Matriz de Pruebas de Network Policies	32
3.2.	Resultados de las Pruebas de Seguridad por CNI	32
3.3.	Prueba de Overhead de Seguridad	33
4.	Pruebas de Eficiencia Computacional — Consumo del Agente CNI	34
5.	Validación del Modelo Matemático MCDA — Objetivo Específico 3	34
5.1.	Validación del Algoritmo IQR	34
5.2.	Validación de la Normalización Min-Max	35
5.3.	Validación del Scoring MCDA por Perfil	36
6.	Validación Funcional del Prototipo Recomendador — Objetivo Específico 4	36
6.1.	Pruebas de Corrección Funcional	36
6.2.	Prueba de Integridad de Datos End-to-End	36
7.	Análisis Consolidado de Resultados	36
7.1.	Respuesta a las Preguntas de Investigación	36
7.2.	Lecciones Aprendidas de las Pruebas	38
8.	Tabla Resumen de Cumplimiento de Objetivos	38
9.	Discusión	39
1.	Síntesis de Resultados Experimentales	39
2.	Comparación con la Literatura	39
3.	Discusión del Modelo MCDA	39
4.	Impacto en la Reducción del Sobredimensionamiento	40
10.	Conclusiones	41
11.	Recomendaciones	42

12.Trabajos Futuros	43
1. Extensión del Framework a Entornos Edge y 5G	43
2. Incorporación de Inteligencia Artificial para Selección Dinámica de CNI . .	43
3. Evaluación de Nuevas Tecnologías CNI y Service Meshes	43
4. Estandarización de la Metodología de Evaluación	44
5. Evaluación del Impacto Energético y Sostenibilidad	44
6. Integración con Sistemas SIEM para Análisis de Seguridad	44
Referencias	46
Apéndices	47
Anexos	49

Lista de tablas

7.1. CNI evaluados: tecnología base, versión desplegada y enfoque.	12
7.2. Variables telemáticas capturadas por el sistema de medición.	14
7.3. Matriz de pesos del modelo MCDA por perfil de usuario.	15
7.4. Casos de uso de seguridad implementados por CNI.	16
7.5. Trazabilidad entre problema de investigación, objetivo y decisión de diseño.	18
7.6. Archivos Terraform del módulo de aprovisionamiento y su función específica.	19
7.7. Instalación de CNIs: método y configuración clave en <code>user_data/ks3_server_init.sh</code>	20
7.8. Métricas de QoS capturadas por el banco de pruebas y su relevancia telemática.	24
7.9. Casos de prueba del pipeline de validación de Network Policies.	26
7.10. Estructura de los tres repositorios del proyecto y su relación con los objetivos específicos.	27
7.11. Desafíos técnicos encontrados durante la implementación y soluciones adoptadas.	27
8.1. Parámetros del escenario de referencia (baseline) para todas las pruebas.	29
8.2. Resultados de throughput TCP por CNI (promedio de 5 corridas de 300 segundos).	31
8.3. Resultados de latencia TCP por CNI (30 muestras $\times$ 5 corridas, outliers IQR eliminados).	32
8.4. Matriz completa de pruebas de seguridad con Network Policies.	32
8.5. Resultados de pruebas de seguridad por CNI.	33
8.6. Impacto de las Network Policies en el rendimiento de red de cada CNI.	33
8.7. Consumo computacional del agente CNI bajo estrés de red ($\sim$ 9 Gbps de tráfico TCP).	34
8.8. Validación del algoritmo IQR con outlier controlado inyectado artificialmente.	35
8.9. Validación de la normalización min-max de las seis métricas del modelo MCDA.	35
8.10. Scores MCDA finales por CNI y perfil ($\star$ indica recomendación del sistema para ese perfil).	36
8.11. Resultados de pruebas de corrección funcional del prototipo SPA.	37
8.12. Respuestas a las preguntas de investigación basadas en evidencia de pruebas.	37

8.13. Cumplimiento de objetivos específicos del proyecto basado en evidencia de pruebas. 38

Índice de figuras

Capítulo 1

Introducción

Esta sección se completará en la siguiente fase.

Capítulo 2

Estado del Arte

Esta sección se completará en la siguiente fase.

Capítulo 3

Planteamiento del Problema

Esta sección se completará en la siguiente fase.

Capítulo 4

Justificación

Esta sección se completará en la siguiente fase.

Capítulo 5

Objetivos

Esta sección se completará en la siguiente fase.

Capítulo 6

Marco Teórico - Conceptual

1. El modelo de red de Kubernetes

Kubernetes no implementa por sí mismo la conectividad entre cargas de trabajo: define un modelo de red y delega su realización en componentes externos. Comprender ese modelo es condición previa para interpretar las métricas de este trabajo, ya que el plano de red —y en particular el plugin que lo implementa— determina el rendimiento, la seguridad y el consumo de recursos del clúster [1].

El modelo de red de Kubernetes establece tres propiedades fundamentales: cada Pod recibe una dirección IP propia, los Pods se comunican entre sí sin traducción de direcciones, y los agentes de cada nodo alcanzan a todos los Pods del clúster. Sobre estas garantías se construyen las abstracciones de descubrimiento, balanceo y control de acceso descritas a continuación [1].

1.1. Identidad de red del Pod

Kubernetes adopta el modelo *IP-por-Pod*: cada Pod obtiene su propia dirección IP y se comunica directamente con otros Pods sin traductores intermedios. Dado que la identidad del Pod es efímera —las réplicas se crean, destruyen y sustituyen de manera dinámica—, la plataforma requiere abstracciones estables para el descubrimiento y el direccionamiento de los destinos [1].

1.2. Nodos y reenvío de paquetes

Cada nodo hospeda múltiples Pods y aporta CPU, memoria y conectividad. En este nivel se instalan las rutas y los mecanismos de reenvío que garantizan la entrega de paquetes al Pod de destino, conforme al modelo de red del clúster: alcance entre Pods, ausencia de superposición de rangos de direcciones y conectividad *east-west* [1]. El agente del plugin de red se ejecuta precisamente en este nivel; por ello, su consumo de CPU y memoria por nodo Worker constituye una de las métricas centrales de la comparativa.

1.3. Direccionamiento estable: Services y EndpointSlices

Un Service expone un identificador estable —por ejemplo, una *ClusterIP* o dirección IP virtual (VIP)— que representa a un conjunto de Pods equivalentes. El cliente se conecta a ese identificador y la plataforma balancea la solicitud hacia una de las réplicas disponibles, desacoplando a los consumidores de los cambios en el ciclo de vida de los Pods [2, 3]. Para mantener actualizada la relación de destinos, Kubernetes emplea *EndpointSlices*, que enumeran las direcciones y puertos de los Pods que respaldan cada Service y reducen la sobrecarga de control frente a los *Endpoints* tradicionales [4]. Esta capa es relevante para interpretar la latencia medida: el tiempo en que una solicitud alcanza el Pod correcto comprende tanto la resolución de la VIP como el reenvío realizado por el plugin de red.

1.4. Ingreso de tráfico externo

El tráfico procedente de Internet no accede directamente a los Pods, sino a través de una capa de entrada. La *Gateway API* formaliza los roles asociados —infraestructura, operador y equipo de aplicación— y las reglas de capa 7 —host, rutas y TLS— que determinan el encaminamiento de cada solicitud hacia el Service correspondiente [5]. En numerosos despliegues, esta capa se encuentra precedida por un balanceador de carga externo de capa 3/4; diseños de alto rendimiento como *Maglev* muestran cómo el *hash* consistente y el seguimiento de flujos permiten una distribución estable y resiliente antes de que el tráfico alcance el clúster [6].

1.5. Aislamiento lógico: Namespaces y NetworkPolicies

Un *Namespace* es una partición lógica del clúster que organiza y nombra recursos por equipo o entorno (*dev*, *test*, *prod*) y evita colisiones de nombres sin multiplicar clústeres; no constituye, sin embargo, una barrera de tráfico [1]. La unicidad de nombres opera dentro del namespace, por lo que pueden coexistir dos servicios homónimos en namespaces distintos; el nombre DNS completo de un servicio incluye el namespace (`<servicio>.<namespace>.svc.cluster.local`), y su uso evita ambigüedades de enrutamiento [1, 3].

Por defecto, la conectividad *east-west* entre namespaces permanece abierta. La restricción del tráfico se especifica mediante *NetworkPolicies* —definidas con *labels* y puertos—, cuya aplicación efectiva depende de que el plugin de red las implemente en su plano de datos. Este mecanismo materializa el principio de *Zero Trust*: conceder únicamente los flujos necesarios y verificar explícitamente cada acceso [7–10].

1.6. Recorrido de una solicitud

El trayecto de una solicitud externa integra las abstracciones anteriores:

1. El cliente externo alcanza la capa de entrada (Ingress/Gateway), opcionalmente a través de un balanceador de carga previo [5].

2. La capa de entrada aplica reglas de capa 7 y dirige la solicitud al Service correspondiente [2, 3].
3. El Service consulta sus EndpointSlices y selecciona un Pod activo [4].
4. El plugin de red encamina el tráfico hasta el Pod de destino en su nodo [1, 7, 11].

En síntesis, Kubernetes define identidad (IP-por-Pod), direccionamiento estable (Services y EndpointSlices), organización (Namespaces), control de acceso (NetworkPolicies) e ingreso gobernado (Gateway API); no obstante, la conectividad subyacente que sostiene todas estas abstracciones la provee el plugin de red, objeto de estudio de este trabajo [1–5, 7, 11].

2. Container Network Interface (CNI)

2.1. Especificación y funciones núcleo

El *Container Network Interface* (CNI) es la especificación —y el conjunto de *plugins* que la implementan— que habilita la conectividad de red de los Pods en Kubernetes. La especificación define cómo el *runtime* de contenedores solicita a un plugin que conecte un Pod a la red y cómo libera los recursos al eliminarlo, mediante operaciones encadenables (ADD y DEL) que permiten combinar responsabilidades, por ejemplo un plugin para IPAM y otro para el mapeo de puertos [7, 11–13]. Sus funciones núcleo son tres:

- **Identidad de red del Pod.** Asignar a cada Pod su dirección IP propia y garantizar la alcanzabilidad *east-west* [1].
- **Conexión y enrutamiento.** Crear interfaces virtuales, instalar rutas y coordinar con el *host* el tránsito del tráfico hacia y desde cada Pod [7, 11].
- **Gestión de direcciones (IPAM).** Reservar y entregar direcciones desde rangos predefinidos, evitando colisiones [11, 13].

La elección del CNI y de su *dataplane* condiciona la latencia, el *throughput* y el consumo de CPU del clúster, con efecto directo sobre el sobredimensionamiento de recursos. Por tanto, su selección informada constituye un problema de ingeniería con consecuencias económicas y de seguridad medibles [14–17].

2.2. Modelos de dataplane: overlay y ruteo nativo

El *dataplane* de un CNI sigue habitualmente uno de dos enfoques. En el modo *overlay*, el tráfico entre nodos se encapsula —por ejemplo, mediante VXLAN sobre UDP— para transportar paquetes de Pods sin modificar la red física; ofrece simplicidad operativa y

desacopla el direccionamiento del clúster respecto del *underlay*, a costa de cabeceras adicionales, ajustes de MTU y cierta carga de CPU por encapsulación y descapsulación. En el modo de ruteo nativo, los Pods son alcanzables mediante rutas de capa 3 entre nodos, sin túneles; suele reducir la latencia y el consumo de CPU, pero exige una integración explícita con la red subyacente y un plan de direccionamiento coherente. La implementación concreta del *fast path* —conmutación basada en OVS o programas eBPF— condiciona adicionalmente el rendimiento y la observabilidad del plano de datos [14–17].

En este trabajo, los cuatro CNI evaluados operan en modo *overlay* sobre la VPC de DigitalOcean —Flannel, Calico y Cilium mediante VXLAN, y Antrea mediante GENEVE—, lo que acota las diferencias de encapsulación al dataplane de cada plugin y no a la topología subyacente. La evaluación se restringe al modo *overlay* por ser el único compatible con la VPC de DigitalOcean sin modificar el *underlay*; la comparación con ruteo nativo se plantea como línea de trabajo futuro.

2.3. Gestión de direcciones (IPAM)

La identidad de red del Pod se materializa en una dirección IP asignada por el mecanismo de *IP Address Management* (IPAM) del CNI. Las estrategias de asignación pueden ser por nodo —cada nodo administra un rango local— o globales —un *pool* común—, con distintas implicaciones en simplicidad operativa y aprovechamiento de direcciones. El plugin *host-local* ilustra el enfoque por nodo: asigna direcciones desde ficheros locales, garantizando unicidad sin dependencias externas. Un plan de direccionamiento bien definido —rangos, *leases*, reutilización y limpieza— minimiza colisiones, facilita *dual-stack* y reduce incidencias durante el escalado o la rotación de Pods [11, 13].

2.4. MTU y fragmentación en redes con encapsulación

La encapsulación propia de los *overlays* reduce la MTU efectiva, ya que parte de cada paquete se destina a cabeceras adicionales. Si la MTU no se ajusta, pueden aparecer fragmentación, caídas de *throughput* y aumentos de latencia. La práctica recomendada consiste en calcular la MTU efectiva restando el *overhead* del túnel y validar el ajuste mediante pruebas controladas, procedimiento especialmente relevante en despliegues con VXLAN o cifrado a nivel de red [18].

2.5. Observabilidad del plano de datos

La operación y optimización de la red del clúster requiere visibilidad sobre los flujos de tráfico: origen, destino, puertos y volúmenes. Las soluciones de *flow visibility* a nivel CNI recolectan y agregan registros de flujo y los correlacionan con identidades lógicas —namespaces y *labels*—, lo que facilita detectar rutas críticas, validar políticas y priorizar mejoras. En arquitecturas de microservicios, esta observabilidad de red complementa las métricas de aplicación para diagnosticar dependencias inesperadas [19, 20].

2.6. Métricas de rendimiento y sobredimensionamiento

La evaluación de un CNI debe considerar la latencia —promedio y colas P95/P99—, el *throughput* sostenido y el costo de CPU por conexión o paquete, fijando el modo del dataplane y el perfil de tráfico para garantizar comparaciones reproducibles. La literatura reporta patrones consistentes entre modos de dataplane y entre *fast paths*, con variaciones significativas en entornos de recursos limitados o de alta tasa de conexión [14–17, 21, 22].

Estas métricas tienen una consecuencia económica directa: un CNI adecuadamente seleccionado y sintonizado reduce los recursos aprovisionados por precaución. Tres prácticas resultan especialmente efectivas: seleccionar el dataplane según el entorno, ajustar la MTU para evitar fragmentación, y automatizar *NetworkPolicies* bajo el principio de *Zero Trust* para limitar el tráfico innecesario. La observabilidad de flujos cierra el ciclo de mejora continua y permite sostener los objetivos de nivel de servicio (SLO) con menor consumo de CPU y memoria [8, 9, 14, 15, 18, 19].

Capítulo 7

Metodología y Diseño de la Solución

En este capítulo se expone la metodología empleada y el diseño detallado e implementación de la solución planteada. Se describen los modelos y algoritmos, las técnicas de medición, la infraestructura como código, la observabilidad, y las estrategias de seguridad implementadas en el clúster Kubernetes.

1. Diseño de la Solución

El diseño responde directamente a las causas identificadas en el árbol de problemas: la ausencia de criterios objetivos para seleccionar el componente de red (CNI), la falta de visibilidad sobre el tráfico interno del clúster y el sobredimensionamiento de recursos que resulta de operar sin datos reales [23]. La solución se estructura en cuatro pilares —una arquitectura de red controlada, un sistema de medición de Calidad de Servicio (QoS), un modelo de decisión multicriterio (MCDA) y una estrategia de seguridad automatizada—, de modo que cada componente posee una justificación técnica derivada del problema de investigación y no constituye un elemento arbitrario de la solución.

1.1. Arquitectura de Red y Topología

El entorno de pruebas se diseñó para que el CNI sea la única variable entre ciclos de medición; el hardware, el sistema operativo, la topología y la duración de las pruebas permanecen constantes en todos los escenarios.

1.1.1. Red underlay

La red underlay constituye la infraestructura física sobre la que opera el clúster. Se optó por la nube pública de DigitalOcean en lugar de servidores físicos propios, ya que la homogeneidad del entorno elimina el ruido de hardware local —variabilidad de NIC, temperatura, interferencias— y garantiza que cualquier diferencia medida en throughput, latencia o CPU sea atribuible al CNI y no al sustrato [1]. El aprovisionamiento se realizó mediante Terraform, herramienta de Infraestructura como Código (IaC): la configuración

completa de los servidores queda descrita en archivos versionados, lo que permite reproducir el entorno de forma idéntica. Esta reproducibilidad es una condición necesaria para la validez del experimento, pues sin ella la comparación entre CNIs pierde valor probatorio [14].

Se utilizó K3s —distribución liviana de Kubernetes— con un nodo Master, datastore PostgreSQL gestionado por DigitalOcean y dos nodos Worker dedicados. El menor *foot-print* de K3s frente a Kubernetes estándar deja una mayor proporción de CPU y RAM disponible para el CNI, lo que aumenta la sensibilidad de las mediciones; el datastore externo elimina el punto único de fallo del plano de control sin la complejidad operativa de un clúster etcd. La topología incluye una red privada VPC para el tráfico interno entre nodos, de manera que la latencia medida dependa exclusivamente de la eficiencia del CNI y no de la variabilidad de Internet público [15].

1.1.2. Red overlay y CNI evaluados

La red overlay, implementada por el plugin CNI, provee la conectividad entre cargas de trabajo distribuidas en distintos nodos. Kubernetes no incluye esta funcionalidad por defecto y delega la elección del CNI en cada organización [7, 11]; esa libertad de elección, ejercida sin criterios objetivos basados en mediciones, es el problema central que aborda este trabajo.

El núcleo del diseño es la comparativa de cuatro CNI con filosofías de implementación distintas (Tabla 7.1).

Cuadro 7.1: CNI evaluados: tecnología base, versión desplegada y enfoque.

CNI	Tecnología Base	Versión	Enfoque
Flannel	VXLAN (L2)	nativo K3s	Overlay sencillo de instalar; sin soporte nativo de NetworkPolicies.
Calico	VXLAN (DO VPC)	3.29.1 Tigera Operator	Tigera Operator con encapsulation: VXLAN ; políticas avanzadas L3/L4.
Cilium	eBPF + VXLAN tunnel	1.16.5	routingMode=tunnel , tunnelProtocol=vxlan ; opera en el kernel con eBPF y filtrado L7 HTTP nativo.
Antrea	OVS (SDN)	2.2.0	Open vSwitch con sistema jerárquico de Tiers (Emergency > SecurityOps > NetworkOps > Platform > Application).

Las tecnologías base de estos CNI sustentan dataplanes diferentes. **VXLAN** crea redes virtuales sobre redes físicas existentes encapsulando los paquetes originales dentro de paquetes UDP, lo que permite transportar tráfico de capa 2 sobre redes de capa 3; el costo es un *overhead* de 50 bytes por paquete, que puede reducir el throughput efectivo en transferencias de paquetes pequeños [18]. **eBPF** permite ejecutar programas de forma segura

dentro del kernel de Linux; aplicado a redes, posibilita que Cilium intercepte y redirija paquetes sin copiarlos al espacio de usuario, eliminando una fuente principal de latencia [16]. **Open vSwitch (OVS)** es un switch virtual orientado a SDN [17] que Antrea emplea para implementar conectividad y políticas mediante un sistema de Tiers jerárquico, capaz de expresar reglas corporativas con mayor prioridad que las NetworkPolicies estándar de Kubernetes.

La selección de estos cuatro CNI no es arbitraria: cada uno aporta un punto de comparación específico y un riesgo característico a medir. Flannel actúa como línea base de conectividad sencilla, con el riesgo de carecer de *enforcement* nativo de NetworkPolicies. Calico aporta madurez operativa y políticas de red en producción, con un posible costo por la evaluación de reglas. Cilium representa el uso de eBPF como mecanismo moderno de aceleración, visibilidad y políticas L7, a cambio de mayor complejidad conceptual y mayor consumo de RAM. Antrea aporta una aproximación basada en OVS y conceptos de SDN, con dependencia de componentes adicionales. En conjunto, cubren el espectro de opciones que enfrenta un equipo de operaciones real, desde la simplicidad operativa hasta las capacidades avanzadas de observabilidad y seguridad.

1.2. Sistema de Medición de Calidad de Servicio (QoS)

La Calidad de Servicio (QoS) cuantifica el comportamiento real de la red desde la perspectiva de las aplicaciones. El sistema de medición sigue los principios de la metrología de redes —control de variables, reproducibilidad y representatividad estadística— para que las diferencias observadas sean atribuibles al CNI y no al procedimiento de medición [20].

1.2.1. Inyección de tráfico

El tráfico se genera con iperf3, herramienta de referencia para pruebas de rendimiento de redes, que opera con un modelo cliente-servidor en el que un Pod emisor envía tráfico a un Pod receptor y ambos registran estadísticas detalladas de la transferencia. Para asegurar tráfico inter-nodo, el cliente y el servidor se separan mediante `podAntiAffinity` suave (`preferredDuringSchedulingIgnoredDuringExecution`, `weight 100`): con dos Workers disponibles, el scheduler los ubica en nodos distintos en la práctica, y la modalidad `preferred` —en lugar de `required`— mantiene la resiliencia ante reinicios de nodo. La ejecución se automatiza mediante CronJobs de Kubernetes, con ráfagas de tráfico TCP de 300 segundos cada 30 minutos, frecuencia que captura el comportamiento de la red en distintos momentos y compone un perfil estadístico representativo [14].

1.2.2. Variables telemáticas capturadas

El sistema de medición parte del principio de que una métrica aislada no describe la calidad de una red. La Tabla 7.2 resume las variables capturadas, que combinan rendimiento de red, estabilidad y costo computacional; la latencia se registra en sus valores

mínimo, promedio, máximo y de variabilidad (*jitter*), de modo que los picos no queden ocultos por el promedio.

Cuadro 7.2: Variables telemáticas capturadas por el sistema de medición.

Variable QoS	Unidad de medida	Relevancia
Throughput (ancho de banda efectivo)	Gbps / Mbps	Capacidad efectiva de transferencia entre Pods. Herramienta: iperf3 -J, 300 s TCP.
Latencia de establecimiento TCP	ms (min/avg/max/mdev)	Costo de iniciar una comunicación entre servicios. Herramienta: script Bash con <code>nc -z -w 2</code> , 30 muestras por corrida.
Retransmisiones TCP	número de paquetes	Indicador de inestabilidad en el canal de encapsulamiento del CNI.
Costo computacional del CNI	% CPU / MB RAM	Recursos que consume el propio CNI para procesar el tráfico; determina el impacto en el sobredimensionamiento.

1.2.3. Pila de observabilidad: Prometheus y Grafana

Prometheus recolecta métricas mediante un modelo de *scraping* periódico sobre cada nodo, registrando estadísticas de CPU, memoria, red y estado del CNI. El almacenamiento en series temporales permite correlacionar el consumo de recursos del agente CNI con los instantes exactos de las pruebas de throughput, verificando que las métricas de eficiencia reflejen el comportamiento bajo carga y no en reposo [20]. **Grafana** constituye la capa de visualización: presenta en dashboards interactivos el throughput, la latencia, el consumo de CPU del agente y las retransmisiones TCP. Esta visibilidad permitió detectar anomalías durante las sesiones —picos de CPU o reinicios inesperados del agente— antes de que comprometieran la integridad de las corridas [20].

1.3. Modelo de Decisión Multicriterio (MCDA)

Para resolver la selección objetiva del CNI se diseñó un modelo de Análisis de Decisión Multicriterio (MCDA), enfoque empleado en ingeniería y economía cuando una decisión involucra múltiples factores con distinta importancia [21]. A diferencia de una comparación basada en un único criterio —por ejemplo, el throughput máximo—, el MCDA captura el *trade-off* entre velocidad, eficiencia de recursos y capacidades de seguridad, y produce una recomendación que refleja las prioridades de cada tipo de organización.

1.3.1. Limpieza estadística y normalización

Antes de aplicar el modelo, los datos recolectados pasan por una limpieza estadística mediante el Rango Intercuartílico (IQR), que elimina *outliers* causados por eventos externos como mantenimientos del proveedor o picos de uso compartido. Las métricas se normalizan después con min-max a una escala común de 0 a 1, considerando el sentido

de optimización de cada variable: en throughput, un valor mayor es mejor; en latencia, CPU, memoria y retransmisiones, un valor menor es preferible, por lo que la escala se invierte. Este procesamiento evita que la recomendación dependa de valores atípicos o de la magnitud numérica de una métrica, y se implementa en el archivo `docs/procesador.js`.

1.3.2. Matriz de pesos por perfil de usuario

Se definieron tres perfiles de usuario basados en patrones comunes de uso de Kubernetes en la industria: Fintech, que prioriza la seguridad; Streaming, que prioriza el rendimiento; e IoT, que prioriza la eficiencia de recursos. Los pesos asignados a cada criterio (Tabla 7.3) son parámetros explícitos del modelo, lo que permite que el prototipo sea ajustable por organización.

Cuadro 7.3: Matriz de pesos del modelo MCDA por perfil de usuario.

Criterio de Evaluación	Fintech (Seguridad)	Streaming (Rendimiento)	IoT (Recursos)
Throughput (velocidad)	15 %	35 %	25 %
Latencia (tiempo de respuesta)	20 %	35 %	20 %
Retransmisiones TCP (estabilidad)	15 %	20 %	15 %
Consumo de CPU del CNI	10 %	10 %	30 %
Consumo de RAM del CNI	10 %	0 %	10 %
Soporte de Network Policies L7	30 %	0 %	0 %
Total	100 %	100 %	100 %

La decisión final se formaliza como una suma ponderada:

$$P_{CNI} = \sum_{i=1}^n (V_i \times W_i)$$

donde P_{CNI} es el puntaje total del plugin, V_i es la valoración normalizada de la métrica i , W_i es el peso asignado y n es el número de métricas evaluadas.

1.4. Seguridad y Micro-segmentación (Zero Trust)

En su configuración inicial, cualquier aplicación dentro del clúster puede comunicarse con cualquier otra. Las Network Policies permiten implementar el principio de *Zero Trust*: conceder solo lo necesario y verificar explícitamente cada acceso [8, 9]. Esto reduce la superficie de ataque, ya que ante el compromiso de un Pod las políticas limitan el movimiento lateral a las rutas explícitamente autorizadas. La aplicación efectiva de estas políticas depende del CNI: si el plugin no las implementa en su plano de datos, la política existe como objeto declarativo sin producir efecto [7, 11].

1.4.1. Estrategia de Default Deny

El diseño implementa Zero Trust en dos niveles: primero, una política de *Default Deny* que bloquea todo el tráfico entrante y saliente de cada namespace; después, una habilitación selectiva que abre únicamente las comunicaciones necesarias. Sobre esta base se definieron tres casos de uso (Tabla 7.4), ejecutados sobre cada CNI que soporta Network Policies —Calico, Cilium y Antrea—; Flannel se incluye como referencia negativa para confirmar la ausencia de *enforcement* en su plano de datos.

Cuadro 7.4: Casos de uso de seguridad implementados por CNI.

Caso	Descripción
<code>zero_trust</code>	Default Deny: bloquea todo ingress y egress por defecto; permite solo el tráfico benchmark explícito.
<code>multi_tier</code>	Reglas explícitas <code>frontend → backend → database</code> mediante label selectors. Cilium agrega CiliumNetworkPolicy L7 HTTP (solo GET/HEAD desde frontend).
<code>egress_block</code>	Bloqueo de egress externo con DNS permitido; tráfico inter-pod interno habilitado.

Dos CNI aportan diferenciadores propios. Cilium implementa filtrado HTTP nativo por método mediante CiliumNetworkPolicy L7 (GET y HEAD permitidos de frontend a backend; POST, PUT y DELETE bloqueados), directamente en eBPF y sin *service mesh*. Antrea, mediante ClusterNetworkPolicy en el Tier `securityops`, impone reglas con prioridad sobre cualquier NetworkPolicy de namespace, modelando políticas corporativas obligatorias —del tipo ISO 27001 o SOC 2— que los desarrolladores no pueden sobrescribir.

1.4.2. Automatización GitOps con ArgoCD

Para preservar la efectividad de las políticas tras cambios en el sistema, se diseñó un flujo GitOps con ArgoCD. Si una política de red se elimina del clúster, ArgoCD detecta la diferencia entre el estado actual y el definido en Git y restaura el recurso automáticamente (`selfHeal: true`); los recursos eliminados del repositorio también se eliminan del clúster (`prune: true`). Esta automatización garantiza la integridad del experimento: durante las rotaciones de CNI, las políticas se restauran a su estado esperado sin intervención manual, lo que elimina una fuente potencial de sesgo en los resultados [9].

1.5. Flujo Integral del Sistema

El flujo completo de datos se organiza en cinco etapas:

1. **Generación de tráfico:** un CronJob activa iperf3-client cada 30 minutos (minutos 0 y 30); la anti-afinidad ubica cliente y servidor en nodos distintos.
2. **Captura de métricas:** iperf3 registra throughput, latencia TCP y retransmisiones (formato JSON, 300 s); un CronJob de latencia ejecuta 30 muestras con `nc -z -w 2` (minutos 10 y 40); Prometheus recolecta CPU y RAM del agente CNI.

3. **Extracción de recursos:** el CronJob del Resource Usage Exporter (minutos 25 y 55) consulta la API HTTP de Prometheus (`/api/v1/query_range`) y genera archivos raw JSON, CSV, PNG y `cni_resource_summary.json`.
4. **Procesamiento y scoring MCDA:** `procesador.js` aplica IQR, normalización min-max y cálculo de puntajes, y genera `cni-data.json`.
5. **Visualización y recomendación:** una SPA en React consume `cni-data.json` y muestra el ranking MCDA por perfil.

1.6. Arquitectura por Capas

La solución se organizó en siete capas para separar responsabilidades: la **capa 1** es la infraestructura underlay (DigitalOcean); la **capa 2**, el clúster Kubernetes K3s en alta disponibilidad; la **capa 3**, el plano CNI (Flannel, Calico, Cilium o Antrea); la **capa 4**, el plano de inyección de tráfico (Pods iperf3 y CronJobs); la **capa 5**, observabilidad y captura (Prometheus, Grafana y exporter de recursos); la **capa 6**, procesamiento y decisión (`procesador.js`: IQR, normalización y MCDA); y la **capa 7**, presentación y recomendación (SPA React). Transversalmente opera la estrategia de seguridad Zero Trust —Network Policies con Default Deny, micro-segmentación y automatización GitOps con ArgoCD—, que asegura que tanto el entorno de prueba como las recomendaciones incorporen consideraciones de seguridad [24].

1.7. Trazabilidad y Requisitos de Diseño

El diseño no se planteó como una arquitectura aislada, sino como respuesta directa al problema formulado en el anteproyecto: las organizaciones adoptan Kubernetes como plataforma de orquestación, pero la calidad final del despliegue depende del plano de red, de la selección del CNI y del control de las comunicaciones internas del clúster [23]. La Tabla 7.5 hace explícita la correspondencia entre cada necesidad, su objetivo asociado y la decisión de diseño que la atiende.

De este planteamiento se derivan cinco requisitos de diseño. La **reproducibilidad** se obtiene mediante infraestructura como código, despliegue declarativo y automatización de experimentos. El **aislamiento de variables** mantiene constante todo elemento distinto del CNI —proveedor, tipo de instancia, versión de Kubernetes, topología y duración—. La **medición integral** evalúa el CNI por rendimiento de red, eficiencia de recursos e impacto de seguridad, y no solo por throughput. La **trazabilidad de la evidencia** exige que cada dato conserve su origen. La **compatibilidad con la operación real** se logra empleando objetos y patrones nativos de Kubernetes —Deployments, CronJobs, ArgoCD— cercanos a la práctica de los equipos de operaciones.

Cuadro 7.5: Trazabilidad entre problema de investigación, objetivo y decisión de diseño.

Necesidad identificada	Objetivo asociado	Respuesta de diseño
Comparar CNIs sin depender de recomendaciones informales.	Evaluar cuantitativamente Calico, Cilium, Flannel y Antrea en escenarios controlados.	Testbed reproducible sobre Kubernetes, ejecución automatizada y consolidación de métricas por CNI.
Controlar el riesgo de comunicaciones internas abiertas.	Evaluar políticas de red seguras y automatizadas.	NetworkPolicies bajo enfoque Zero Trust, casos de denegación por defecto y segmentación por capas.
Convertir resultados técnicos heterogéneos en una decisión comprensible.	Definir criterios, umbrales y ponderaciones para comparación objetiva.	Modelo MCDA con criterios de rendimiento, eficiencia de recursos e impacto de seguridad.
Apoyar a una organización que necesita seleccionar y configurar un CNI.	Implementar un prototipo funcional de recomendación.	Prototipo web que consume datos procesados, aplica perfiles de decisión y explica la recomendación.
Evitar sobredimensionamiento de infraestructura.	Relacionar desempeño de red con consumo de CPU y memoria.	Captura de recursos del CNI mediante Prometheus y cálculo de eficiencia relativa por perfil.

1.8. Prototipo Recomendador

El prototipo recomendador es la capa de interacción entre el modelo matemático y el usuario final. Su salida comprende tres niveles: la recomendación directa (qué CNI se sugiere), la justificación (qué métricas la explican) y la orientación de configuración (qué consideraciones debe atender el usuario al desplegar ese CNI), estructura que responde al objetivo específico 4 del anteproyecto [23]. Se implementó como SPA con React 19, Vite 7 y Tailwind CSS 3; el *build* de producción se despliega en GitHub Pages, y la actualización de datos encadena `procesador.js` —que genera `cni-data.json`— con la inyección de esos datos en el *bundle*.

1.9. Alcance y Límites del Diseño

El diseño se limita a evaluar CNIs en un testbed controlado sobre Kubernetes y no pretende generalizar automáticamente los resultados a cualquier proveedor, versión o hardware. Emplea tráfico TCP controlado como señal principal y no asume que una mayor complejidad implique un mejor resultado. La solución se concibió para sostener una mejora continua: a medida que se incorporen nuevas corridas o versiones de CNIs, el pipeline puede recalcular los puntajes. En lugar de entregar una decisión congelada, ofrece un método para decidir con evidencia actualizable, cuyo beneficio es a la vez técnico —reemplaza opiniones por datos medidos en condiciones reproducibles— y económico, pues conocer el consumo de CPU de cada CNI por unidad de tráfico permite dimensionar la infraestructura sin márgenes precautorios excesivos [23].

2. Implementación de la Solución

Toda la infraestructura está codificada en Terraform, toda la configuración en manifiestos YAML versionados en Git, y toda prueba está automatizada en CronJobs de

Kubernetes —condición necesaria para que los resultados sean reproducibles y atribuibles al CNI [14].

2.1. Aprovisionamiento de Infraestructura con Terraform

El aprovisionamiento de servidores y redes se realizó de forma completamente automatizada mediante Terraform (IaC). Al describir la infraestructura en archivos versionados, Terraform la recrea de manera idéntica en cada ejecución y elimina la posibilidad de que variaciones manuales en el aprovisionamiento introduzcan diferencias entre los entornos de cada CNI [14].

2.1.1. Estructura del Código Terraform (K8s-bootstrap/00.bootstrap)

Cuadro 7.6: Archivos Terraform del módulo de aprovisionamiento y su función específica.

Archivo Terraform	Responsabilidad en la Infraestructura
<code>main.tf</code>	Define los recursos principales: Droplets de DigitalOcean y red VPC privada.
<code>variables.tf</code>	Centraliza todos los parámetros configurables: número de nodos (<code>agent_count = 2</code>), tamaño de instancias, región y proveedor CNI.
<code>ssh_keys.tf</code>	Automatiza la gestión de llaves SSH para el acceso seguro entre nodos.
<code>firewall.tf</code>	Configura las reglas de firewall a nivel de nube: primera línea de seguridad perimetral.
<code>outputs.tf</code>	Genera automáticamente el archivo kubeconfig con las credenciales del clúster K3s.

El proceso de aprovisionamiento tiene tres etapas: `terraform plan` (compara estado actual vs. código), `terraform apply` (crea Droplets, VPC y firewall) y `terraform output` (extrae IPs y kubeconfig). Terraform mantiene un archivo `terraform.tfstate` que permite destruir y recrear toda la infraestructura de manera controlada, garantizando la repetibilidad científica: cada CNI se evaluó en condiciones idénticas de arranque.

2.2. Configuración del Clúster K3s en Alta Disponibilidad

Una vez aprovisionados los servidores, se instaló y configuró K3s, una distribución liviana de Kubernetes, con topología de 1 nodo Master + 2 nodos Worker dedicados.

2.2.1. Base de Datos Externa para el Plano de Control

El plano de control se configuró con una base de datos PostgreSQL gestionada de DigitalOcean (también provisionada via Terraform) como datastore externo. Esto elimina la complejidad de mantener un clúster etcd y garantiza un plano de control sin punto único de fallo (SPOF).

2.2.2. Instalación: clúster sin CNI para plugins externos y modo nativo para Flannel

La implementación establece una distinción metodológica deliberada: Flannel se evalúa en su modo de despliegue más habitual —integrado nativamente en K3s con `flannel-iface: eth1`—, mientras que Calico, Cilium y Antrea se instalan como plugins externos sobre un clúster sin CNI. La asimetría refleja las condiciones reales de uso: Flannel es el CNI predefinido de K3s y la mayoría de equipos lo adopta sin configuración adicional, en tanto que los otros tres requieren instalación explícitamente y de forma declarativa. Evaluar cada CNI en su escenario de despliegue típico aumenta la validez ecológica del experimento; debe considerarse, no obstante, que Flannel se beneficia de la integración nativa de K3s —gestión automática de CIDR y *hooks* de kubelet— que los plugins externos deben replicar por su cuenta.

Para los CNIs externos, la configuración crítica es el parámetro `-flannel-backend=none`, que desactiva completamente el CNI integrado de K3s y evita interferencias con el plugin bajo evaluación. Adicionalmente, se configura `-disable-network-policy` para que K3s no intente implementar NetworkPolicies por sí mismo.

La configuración resultante en `/etc/rancher/k3s/config.yaml` para CNIs externos incluye:

```
flannel-backend: none
disable-network-policy: true
```

Cuadro 7.7: Instalación de CNIs: método y configuración clave en `user_data/ks3_server_init.sh`.

CNI	Método de instalación	Configuración clave
Flannel	Nativo K3s	<code>flannel-iface: eth1</code> (interfaz VPC de DigitalOcean)
Calico	Tigera Operator v3.29.1	<code>encapsulation: VXLAN</code> , <code>CIDR 10.42.0.0/16</code> , <code>nodeAddressAutodetectionV4.interface: eth1</code>
Cilium	Helm 1.16.5	<code>routingMode=tunnel</code> , <code>tunnelProtocol=vxlan</code> , <code>ipam.mode=kubernetes</code> , <code>devices=eth1</code> , <code>operator.replicas=1</code>
Antrea	Manifest v2.2.0	<code>transportInterface: eth1</code> , <code>serviceCIDR: 10.43.0.0/16</code>

El protocolo de rotación de CNIs tuvo cuatro pasos: (1) `kubect1 delete` para eliminar recursos del CNI actual; (2) verificación de que no quedan interfaces de red anteriores en los nodos; (3) reinicio de los nodos Worker para limpiar estado en memoria; (4) aplicación del manifiesto del nuevo CNI via ArgoCD. Este protocolo garantiza que cada CNI inicia sobre un nodo completamente limpio, sin residuos de configuraciones anteriores que pudieran contaminar las métricas del siguiente ciclo de pruebas.

2.3. Orquestación y Entrega Continua: GitOps con ArgoCD

Con el clúster K3s funcionando y sin CNI, el siguiente componente instalado fue ArgoCD. ArgoCD implementa el paradigma GitOps: el repositorio Git es la única fuente de

verdad sobre el estado del clúster. Todo lo que debe existir en el clúster está definido en archivos YAML dentro del repositorio `K8s-Tesis-Apps`. Esta centralización en Git facilita la auditoría del experimento: cualquier cambio en la configuración queda registrado con autor, fecha y motivo, haciendo el proceso de prueba completamente trazable.

2.3.1. El Patrón App of Apps

La implementación de ArgoCD utiliza el patrón App of Apps: existe UNA sola aplicación raíz (ubicada en `K8s-bootstrap/30.apps-of-apps-install`) que ArgoCD conoce directamente, y esa aplicación raíz contiene las definiciones de todas las demás aplicaciones (CNIs, stack de métricas, benchmarks, políticas de red).

2.3.2. Configuración de `selfHeal` y `prune`

selfHeal: true — Ante la modificación manual de un recurso del clúster —por ejemplo, la eliminación de una `NetworkPolicy`—, ArgoCD detecta la diferencia con el repositorio Git y restaura el estado original, lo que resultó crítico para la integridad del experimento.

prune: true — Al eliminar un archivo YAML del repositorio Git, ArgoCD elimina también el recurso correspondiente del clúster, lo que evita la acumulación de recursos huérfanos.

```
# Definición de la Application raíz en ArgoCD (App of Apps)
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: apps-of-apps
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/equipo/K8s-Tesis-Apps
    targetRevision: main
    path: apps/
  syncPolicy:
    automated:
      selfHeal: true # Restaura cambios manuales automáticamente
      prune: true # Elimina recursos borrados del repo
```

2.3.3. Overlays de Kustomize para los CNIs

Cada CNI tiene una configuración ligeramente diferente. Para gestionarlas sin duplicar código, se utilizó Kustomize con el patrón base/overlay: existe una configuración base con los parámetros comunes, y sobre esa base se aplican overlays específicas para cada CNI.

Las apps gestionadas por ArgoCD incluye: `cni-test-iperf` (benchmarks de throughput y latencia), `cni-resource-exporter` (exportación de recursos desde Prometheus), `cni-np-test` (pruebas de Network Policies, creada solo cuando `cni_provider != "flannel"` y el caso de uso está definido), y `metrics` (Prometheus + Grafana).

2.4. Implementación del Stack de Observabilidad

El stack de observabilidad se implementó mediante el `kube-prometheus-stack` (Helm chart) que instala Prometheus, Grafana y exportadores preconfigurados para Kubernetes.

2.4.1. Prometheus: El Recolector de Métricas

Prometheus opera con un modelo pull: visita periódicamente los endpoints de métricas (cada 15 segundos en la configuración del proyecto). Para que Prometheus sepa qué métricas recolectar de los agentes CNI, se implementaron `ServiceMonitors` (CRD), apuntando específicamente al endpoint de métricas de cada agente. Un intervalo de 15 segundos proporciona suficiente granularidad para detectar picos de consumo durante las ráfagas de tráfico iperf3, sin generar una carga de scraping que interfiera con las métricas bajo observación.

Las métricas clave recolectadas del agente CNI incluyen:

- `container_cpu_usage_seconds_total` (filtrado por pod del agente): costo computacional del CNI (criterio C2 del marco MCDA).
- `container_memory_rss` (filtrado por pod del agente): memoria física real del proceso del agente CNI.
- `node_network_transmit_bytes_total` / `node_network_receive_bytes_total`: bytes totales de red por nodo.
- `kube_pod_info` + `kube_pod_status_phase`: estado del agente CNI para detectar reinicios durante las pruebas.

2.4.2. Grafana: Los Dashboards de Red

Grafana centraliza la lectura operativa de las series de Prometheus. Sus paneles agrupan throughput, latencia, consumo del agente CNI y score MCDA. Esta correlación vincula ráfagas iperf3 con CPU, RSS y el ranking calculado por `procesador.js`.

2.4.3. Resource Usage Exporter: Extracción Automatizada de Datos

El `cni-resource-usage-exporter` corre como CronJob en `K8s-Tesis-Apps/grafana-exporter`. ArgoCD sincroniza el recurso mediante la app `cni-resource-exporter`. El CronJob:

- Ejecuta la extracción en los minutos 25 y 55 de cada hora, tras cada ventana de benchmarks.
- Consulta directamente la API HTTP de Prometheus (`/api/v1/query_range`) — no interactúa con Grafana.
- Recolecta CPU % por nodo, RAM % por nodo, RAM usada, CPU cores y RAM del agente CNI.

- Genera raw/, csv/, images/ y cni_resource_summary.json por corrida.
- Persiste cada salida con timestamp en K8S-CNI-Results/results/cni-benchmarks/{cni}/resource_usage_nodes/.

2.5. Implementación del Banco de Pruebas de QoS

2.5.1. Escenario de Rendimiento con iperf3

El banco de throughput se define en K8s-Tesis-Apps/cni_test_iperf:

El iperf3-server (Deployment): Un Deployment ejecuta una réplica de iperf3 en modo servidor. Un Service ClusterIP expone un nombre DNS estable para el cliente.

El iperf3-client (CronJob): La programación 0,30 * * * * activa el cliente en los minutos 0 y 30. **concurrencyPolicy:** Forbid bloquea corridas simultáneas. Cada activación crea un Pod y genera tráfico TCP durante 300 segundos. La salida conserva formato JSON con `iperf3 -t 300 -i 10 -J`. La retención máxima queda en 5 corridas (`RETENTION_MAX_RUNS="5"`).

La anti-afinidad del cliente frente al servidor se implementó como `preferredDuringSchedulingIgnoredDuringExecution` (suave, weight 100), forzando tráfico inter-nodo en la práctica. Esta ubicación garantiza que todo el tráfico de benchmark atravesase el mecanismo de encapsulación del CNI entre nodos, que es exactamente lo que se quiere medir.

```
# iperf-client-cronjob.yaml -- fragmento relevante
schedule: '0,30 * * * *' # Minutos 0 y 30
concurrencyPolicy: Forbid
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchLabels:
              app: iperf-server
          topologyKey: kubernetes.io/hostname
# iperf3 -c iperf-server-svc -t 300 -i 10 -J
```

2.5.2. Cliente de Latencia Personalizado

El CronJob `latency-client-cronjob.yaml` ejecuta la prueba de latencia. Su programación corre en los minutos 10 y 40 de cada hora. Esa separación evita solapamiento con el inicio del throughput.

El cliente registra TCP Connect con `nc -z -w 2` y timestamp en ms. Cada corrida toma 30 muestras y calcula min/avg/max/mdev. El JSON normalizado conserva campos de latencia, muestras intentadas y muestras exitosas.

Cuadro 7.8: Métricas de QoS capturadas por el banco de pruebas y su relevancia telemática.

Métrica Capturada	Unidad	Relevancia para QoS Telemático
Throughput TCP sostenido	Gbps / Mbps	Capacidad efectiva de la red.
Latencia mínima TCP connect	ms	Overhead mínimo del CNI sin congestión.
Latencia promedio TCP connect	ms	Comportamiento típico de solicitudes.
Latencia máxima / Jitter	ms	Peor caso y variabilidad. Crítico para tiempo real.
Retransmisiones TCP	count / sesión	Indicador de problemas en encapsulamiento o congestión.
CPU del agente CNI	millicores (m)	Impacto directo en costo de infraestructura cloud.
RAM del agente CNI (RSS)	MiB	Huella de memoria del agente CNI por nodo.

2.6. Procesamiento Estadístico y Aplicación del Modelo MCDA

2.6.1. Limpieza con el Rango Inter cuartílico (IQR)

`docs/procesador.js` transforma datos crudos en datos consolidados. El script descubre los CNIs disponibles bajo `results/cni-benchmarks`. Luego extrae métricas desde JSON y CSV. Finalmente aplica IQR:

```
// procesador.js -- Funcion de limpieza estadistica IQR
function calcularEstadisticasLimpias(valores) {
  const sorted = [...valores].sort((a, b) => a - b);
  const q1 = sorted[Math.floor(sorted.length * 0.25)];
  const q3 = sorted[Math.floor(sorted.length * 0.75)];
  const iqr = q3 - q1;
  const lowerBound = q1 - 1.5 * iqr;
  const upperBound = q3 + 1.5 * iqr;
  const valoresLimpios = valores.filter(
    v => v >= lowerBound && v <= upperBound);
  const promedio = valoresLimpios.reduce(
    (sum, v) => sum + v, 0) / valoresLimpios.length;
  return { promedio,
    outliersPorcentaje: ((valores.length - valoresLimpios.length)
      / valores.length * 100).toFixed(1)
  };
}
```

El pipeline descarta valores atípicos antes de calcular promedios. Dado que un evento cloud anómalo puede sesgar una corrida, el filtro estabiliza el puntaje MCDA del CNI.

2.6.2. Normalización y Cálculo del Score MCDA

Después de la limpieza, el pipeline aplica normalización min-max. Cada valor se escala al rango $[0, 1]$. En métricas positivas, como throughput, la normalización es directa. En métricas negativas, como latencia o CPU, la escala se invierte.

Cuando la métrica penaliza, el valor más bajo recibe 1 y el más alto recibe 0. La escala común compara Gbps, milisegundos y millicores sin sesgo por magnitud numérica.

```
// procesador.js -- Calculo del score MCDA por perfil
const perfiles = {
  fintech: { throughput: 0.15, latencia: 0.20,
    retransmisiones: 0.15, cpu: 0.10,
    ram: 0.10, networkPoliciesL7: 0.30 },
```

```
streaming: { throughput: 0.35, latencia: 0.35,
             retransmisiones: 0.20, cpu: 0.10,
             ram: 0.00, networkPoliciesL7: 0.00 },
iot: { throughput: 0.25, latencia: 0.20,
       retransmisiones: 0.15, cpu: 0.30,
       ram: 0.10, networkPoliciesL7: 0.00 }
};
```

El pipeline calcula overhead de seguridad cuando detecta `with_network_policy`. En ese caso, compara la métrica base contra la métrica con política activa.

2.7. Prototipo Recomendador: Aplicación Web SPA

2.7.1. Tecnologías de Implementación

La SPA fue implementada con React 19 (gestión de estado por componentes), Vite 7 (bundler; build de producción vía `build:pages` con `-outDir ../docs/recommender`) y Tailwind CSS 3 (estilos utilitarios directamente en JSX, sin CSS personalizado).

`npm run sync:data` ejecuta `procesador.js` y genera `cni-data.json`. Después, `export-spa-data` inyecta los datos en el bundle. `recommendationModel.js` concentra la lógica de recomendación.

2.7.2. Flujo de la Aplicación

1. **Selección de Perfil:** La pantalla principal muestra tres tarjetas: Fintech/Seguridad, Streaming/Rendimiento e IoT/Recursos.
2. **Visualización de Resultados:** La aplicación carga `cni-data.json` y calcula el ranking MCDA para el perfil seleccionado. El CNI con mayor puntaje queda marcado como recomendación.
3. **Justificación Técnica:** La interfaz expone las métricas brutas promedio de cada CNI en gráficas comparativas.

2.8. Implementación de Network Policies y Seguridad Zero Trust

2.8.1. Políticas Default Deny

Default Deny operacionaliza el modelo Zero Trust. La política bloquea todo el tráfico inicial y habilita solo comunicaciones necesarias. Con esta regla, toda comunicación no autorizada falla sin intervención del operador.

El siguiente manifiesto aplica la política a todos los Pods del namespace `production`:

```
# default-deny-all.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
```

```
podSelector: {} # aplica a TODOS los Pods del namespace
policyTypes:
- Ingress # Bloquea todo el trafico entrante
- Egress # Bloquea todo el trafico saliente
```

2.8.2. Micro-segmentación por Capas de Aplicación

Sobre Default Deny, las políticas específicas acotan el tráfico permitido. Las reglas frontend → backend → database usan label selectors.

En Cilium, una CiliumNetworkPolicy de capa 7 admite solo métodos GET y HEAD desde frontend hacia backend. En Antrea, una ClusterNetworkPolicy en el Tier securityops prioriza sobre cualquier NetworkPolicy de namespace.

Esta estratificación evalúa seguridad granular en condiciones reales. También cuantifica si esa granularidad introduce overhead detectable, uno de los criterios del modelo MCDA.

2.8.3. Validación Automatizada de Políticas

Se implementó un pipeline de validación que emplea nc (netcat) dentro de Pods de prueba para verificar la conectividad. Para cada par de Pods, el pipeline define una expectativa —la conexión debe permitirse o debe bloquearse— y reporta un error cuando el comportamiento observado no coincide con ella.

Cuadro 7.9: Casos de prueba del pipeline de validación de Network Policies.

Prueba de Validación	Resultado Esperado y Justificación
frontend → backend (puerto 8080)	PERMITIDO — Comunicación necesaria para el funcionamiento de la aplicación.
frontend → database (puerto 5432)	BLOQUEADO — El frontend no debe acceder directamente a la base de datos.
backend → database (puerto 5432)	PERMITIDO — El backend necesita leer y escribir datos.
database → cualquier Pod	BLOQUEADO — La base de datos nunca debe iniciar conexiones salientes.
Pod externo → namespace production	BLOQUEADO — Sin una regla explícita, ningún Pod externo puede entrar.

2.9. Estructura de Repositorios y Entregables Técnicos

La Tabla 7.10 resume la organización del proyecto en tres repositorios Git.

La arquitectura mantiene un flujo unidireccional. K8s-bootstrap crea infraestructura; K8s-Tesis-Apps despliega aplicaciones y genera métricas. K8S-CNI-Results almacena, procesa y visualiza resultados.

2.10. Desafíos de Implementación y Soluciones Adoptadas

Cuadro 7.10: Estructura de los tres repositorios del proyecto y su relación con los objetivos específicos.

Repositorio	Contenido Principal	Relación con los Objetivos
K8s-bootstrap	Código Terraform (<code>00.bootstrap</code>), scripts de instalación K3s HA (<code>10.k3s-ha-do</code>), manifiestos de ArgoCD (<code>20.argo-cd-install</code>) y App of Apps (<code>30.apps-of-apps-install</code>).	OE1: Provee la infraestructura controlada necesaria para las pruebas.
K8s-Tesis-Apps	Overlays Kustomize para los 4 CNIs, stack de observabilidad (<code>apps/metrics</code>), benchmarks iperf3 (<code>cni_test_iperf</code>), cliente de latencia, Network Policies y Grafana Exporter.	OE1, OE2 y OE3: Implementa las pruebas, la seguridad y el sistema de recolección.
K8S-CNI-Results	Archivos JSON/CSV con resultados por corrida, <code>procesador.js</code> (modelo MCDA) y la SPA recomendadora (<code>cni-recommender-spa</code>).	OE3 y OE4: Implementa el modelo de comparación objetiva y el prototipo.

Cuadro 7.11: Desafíos técnicos encontrados durante la implementación y soluciones adoptadas.

Desafío Encontrado	Solución Técnica Implementada
MTU mismatch al instalar Cilium: Pods no podían comunicarse, paquetes se fragmentaban silenciosamente en la VPC. Flannel y Calico detectan automáticamente la MTU de la interfaz subyacente en sus versiones actuales; Cilium requirió configuración explícita porque su dataplane eBPF no hereda el valor de MTU del sistema operativo sin intervención.	Se calculó la MTU efectiva: $1500 \text{ (VPC)} - 50 \text{ (overhead VXLAN)} = 1450$. Se configuró el parámetro <code>mtu: 1450</code> en el overlay de Kustomize de Cilium.
ArgoCD en estado OutOfSync tras la rotación de CNI: Pods perdían conectividad durante el cambio.	Se implementó una secuencia de rotación segura: poner ArgoCD en modo Suspended, rotar el CNI, verificar conectividad, reactivar ArgoCD.
Outliers frecuentes en pruebas de madrugada: DigitalOcean mostraba throttling de CPU en horas de alta demanda del datacenter.	Se configuró una ventana de pruebas exclusiva para horario laboral. El algoritmo IQR filtra automáticamente estos outliers.
<code>procesador.js</code> necesitaba datos de Network Policies L7 que iperf3 no puede medir (capa 4).	Se añadió una variable binaria (soporta/no soporta) al modelo MCDA para la capacidad de Network Policies L7. Cilium recibe valor 1, los demás 0.

2.11. Flujo completo de ejecución

El flujo completo sigue las etapas descritas anteriormente. Terraform prepara la infraestructura; K3s y ArgoCD habilitan el plano de ejecución. Los CronJobs producen datos, `procesador.js` consolida resultados y la SPA presenta la recomendación.

Capítulo 8

Resultados y Pruebas

Este capítulo documenta los protocolos, escenarios y resultados de las pruebas sobre los cuatro objetivos específicos del proyecto. En cada ciclo se modificó únicamente el CNI bajo evaluación; hardware, sistema operativo, versión de Kubernetes, tamaño de paquetes, duración y condiciones de red permanecieron constantes, lo que hace posible atribuir las diferencias observadas al plugin y no al entorno.

1. Metodología General de Pruebas

1.1. Control de Condiciones: el Escenario de Referencia (Baseline)

Se definió un escenario de referencia con los siguientes parámetros fijos para todos los CNIs:

Parámetro Controlado	Valor Fijo para Todo el Experimento
Tipo de instancia cloud	DigitalOcean Droplet — Ubuntu 22.04 LTS, 2 vCPU, 4 GB RAM para nodos Worker
Versión de Kubernetes	K3s canal stable — la misma versión en todos los ciclos de prueba
Región del datacenter	fra1 (Frankfurt) — mismo datacenter para todos los nodos en todas las pruebas
Red entre nodos	VPC privada DigitalOcean — MTU 1500 bytes, sin tráfico externo
Duración de cada prueba iperf3	300 segundos (5 minutos) por corrida
Número de corridas por CNI	5 corridas independientes — permite calcular promedios y detectar variabilidad
Protocolo de red bajo prueba	TCP — el protocolo dominante en aplicaciones empresariales reales
Herramienta de throughput	iperf3 con salida JSON (-J), un solo stream TCP (-P 1)
Herramienta de latencia	Script Bash con nc -z -w 2 — 30 muestras por corrida

Cuadro 8.1: Parámetros del escenario de referencia (baseline) para todas las pruebas.

1.2. Automatización: de la Prueba Manual a la Prueba Orquestada

Todas las pruebas se orquestaron mediante CronJobs de Kubernetes. La secuencia de ejecución automática en cada ciclo fue:

1. CronJob del cliente iperf3 se activa (minutos 0 y 30). Kubernetes crea el Pod cliente en un nodo diferente al servidor (podAntiAffinity suave, weight 100).
2. Pod cliente ejecuta iperf3 durante 300 segundos y guarda el resultado JSON.
3. CronJob del cliente de latencia ejecuta 30 mediciones de TCP connect (minutos 10 y 40) y guarda su resultado JSON.
4. Al finalizar ambas pruebas, el Resource Usage Exporter (minutos 25 y 55) consulta Prometheus para extraer el consumo de CPU y RAM del agente CNI.
5. `procesador.js` actualiza `cni-data.json` con los nuevos datos procesados.

La orquestación automática elimina la variabilidad humana de la ejecución: cada corrida sigue exactamente la misma secuencia con los mismos intervalos, de modo que las diferencias observadas entre CNIs se deben al plugin y no al operador.

La elección de cinco corridas por CNI responde a un equilibrio entre rigor estadístico y viabilidad operativa. Una sola corrida permitiría que un documento o evento anómalo dominara el resultado, mientras que un número muy elevado extendería el tiempo de prueba sin un beneficio proporcional. Con dosificadas corridas distribuidas en el tiempo, el algoritmo IQR puede identificar y descartar una corrida anómala, y el promedio de las restantes resulta estadísticamente representativo; este criterio es habitual en la investigación de redes sobre entornos cloud [14, 15].

2. Pruebas de Rendimiento de Red — Objetivo Específico 1

2.1. Protocolo de Prueba de Throughput con iperf3

iperf3 establece una conexión entre un proceso servidor y un proceso cliente, y el cliente envía datos lo más rápido que la red permite durante el tiempo configurado. Se usó el parámetro `-P 1` (un solo stream TCP) para representar el comportamiento de una aplicación empresarial típica, no la carga máxima teórica.

```
# Comando iperf3 ejecutado por el CronJob cliente
# -c: servidor -t: 300s -J: JSON -P 1: un solo stream
iperf3 -c iperf-server-svc.cni-benchmark.svc.cluster.local \
  -t 300 -J -P 1 \
  > /results/throughput-$(date +%Y%m%d_%H%M%S)-${CNI_NAME}.json
```

2.2. Resultados de Throughput TCP por CNI

Los resultados presentados son los promedios de las 5 corridas de 300 segundos para cada CNI, después de aplicar el filtro IQR para eliminar outliers.

Métrica	Flannel (VXLAN)	Calico (VXLAN)	Cilium (eBPF)	Antrea (OVS)
Throughput Sender promedio	9.21 Gbps	9.48 Gbps	9.67 Gbps	9.35 Gbps
Throughput Receiver promedio	9.18 Gbps	9.45 Gbps	9.64 Gbps	9.31 Gbps
Retransmisiones TCP (promedio)	12.4 / sesión	4.8 / sesión	2.1 / sesión	7.3 / sesión
Diferencia Sender-Receiver	~0.03 Gbps	~0.03 Gbps	~0.03 Gbps	~0.04 Gbps
Variabilidad entre corridas (σ)	± 0.18 Gbps	± 0.09 Gbps	± 0.07 Gbps	± 0.12 Gbps
Outliers descartados por IQR	1 de 5 (20 %)	0 de 5 (0 %)	0 de 5 (0 %)	1 de 5 (20 %)

Cuadro 8.2: Resultados de throughput TCP por CNI (promedio de 5 corridas de 300 segundos).

Todos los CNIs se aproximan al límite teórico de la red VPC de DigitalOcean (~9.7–9.8 Gbps). Las diferencias en retransmisiones, en cambio, son significativas: Cilium con eBPF genera 2.1 retransmisiones por sesión, frente a las 12.4 de Flannel —casi seis veces más—. Esta diferencia se explica porque eBPF opera directamente en el kernel y resuelve las decisiones de enrutamiento con mayor rapidez que el mecanismo VXLAN de Flannel.

2.3. Protocolo de Prueba de Latencia TCP

La latencia de establecimiento TCP mide el tiempo que tarda en completarse el handshake de tres vías de TCP, que ocurre cada vez que una aplicación inicia una nueva conexión. El cliente ejecuta `nc -z -w 2` al servicio `iperf3-server`, mide el tiempo en ms y calcula min/avg/max/mdev sobre 30 muestras por corrida. Esta métrica es especialmente relevante para arquitecturas de microservicios, donde el costo de iniciar una conexión puede dominar el tiempo total de respuesta y el throughput del enlace no captura esa penalización.

2.4. Resultados de Latencia TCP por CNI

Los datos de latencia revelan la mayor brecha de rendimiento entre CNIs. Cilium logra una latencia promedio de 0.312 ms, mientras que Flannel alcanza 0.687 ms, el doble. El P99 de Flannel (2.156 ms) significa que el 1 % de las conexiones tardan más de 2 ms en establecerse. En una aplicación que realiza 10.000 conexiones por segundo, ese 1 % representa 100 conexiones lentas por segundo.

Métrica de Latencia	Flannel (VXLAN)	Calico (VXLAN)	Cilium (eBPF)	Antrea (OVS)
Latencia mínima (ms)	0.412 ms	0.287 ms	0.198 ms	0.341 ms
Latencia promedio (ms)	0.687 ms	0.431 ms	0.312 ms	0.512 ms
Latencia máxima (ms)	2.341 ms	0.891 ms	0.543 ms	1.124 ms
Jitter / Desviación estándar (ms)	±0.389 ms	±0.142 ms	±0.087 ms	±0.201 ms
Percentil P95 (ms)	1.834 ms	0.712 ms	0.478 ms	0.893 ms
Percentil P99 (ms)	2.156 ms	0.843 ms	0.521 ms	1.067 ms

Cuadro 8.3: Resultados de latencia TCP por CNI (30 muestras × 5 corridas, outliers IQR eliminados).

3. Pruebas de Seguridad con Network Policies — Objetivo Específico 2

3.1. Matriz de Pruebas de Network Policies

Se diseñó una matriz de pruebas que cubre los escenarios críticos de seguridad en un clúster Kubernetes. La herramienta de verificación principal fue `kubect1 exec` combinada con `nc` (netcat) dentro de Pods de prueba.

Caso de Prueba	Descripción, Herramienta y Criterio de Éxito
TC-SEC-01: Default Deny Ingress	Ningún Pod externo puede enviar tráfico al namespace sin regla explícita. Verificación: <code>nc -zv <pod-ip>8080</code> . ÉXITO: timeout en menos de 5 s.
TC-SEC-02: Default Deny Egress	Los Pods en producción no pueden iniciar conexiones salientes no autorizadas. Verificación: <code>curl -max-time 3</code> . ÉXITO: timeout.
TC-SEC-03: Frontend→Backend permitido	Pod con etiqueta <code>tier=frontend</code> puede conectarse al backend en puerto 8080. ÉXITO: Connection succeeded.
TC-SEC-04: Bloqueo Frontend→Database	Pod frontend NO puede conectarse a la base de datos directamente. ÉXITO: Connection refused o timeout.
TC-SEC-05: Backend→Database permitido	Pod backend SÍ puede conectarse a la base de datos en puerto 5432. ÉXITO: Connection succeeded.
TC-SEC-06: Database NO inicia conexiones	La base de datos nunca inicia conexiones salientes. ÉXITO: Bloqueo por política de Egress.
TC-SEC-07: Bloqueo entre namespaces	Un Pod de namespace staging no puede comunicarse con Pods de production. ÉXITO: Bloqueo confirmado.
TC-SEC-08: GitOps Auto-restauración	Si una política se elimina manualmente, ArgoCD la restaura en menos de 3 minutos. ÉXITO: Política restaurada automáticamente.

Cuadro 8.4: Matriz completa de pruebas de seguridad con Network Policies.

3.2. Resultados de las Pruebas de Seguridad por CNI

El hallazgo más crítico es que Flannel no implementa Network Policies en su plano de datos: las políticas se crean sin error en Kubernetes, pero el tráfico que debería bloquearse

Caso de Prueba	Flannel	Calico	Cilium	Antrea
TC-SEC-01: Default Deny Ingress	NO SOPORTA	PASA	PASA	PASA
TC-SEC-02: Default Deny Egress	NO SOPORTA	PASA	PASA	PASA
TC-SEC-03: Frontend→Backend	N/A	PASA	PASA	PASA
TC-SEC-04: Frontend→DB bloqueado	NO BLOQUEA	PASA	PASA	PASA
TC-SEC-05: Backend→DB permitido	N/A	PASA	PASA	PASA
TC-SEC-06: DB no inicia conexiones	NO SOPORTA	PASA	PASA	PASA
TC-SEC-07: Bloqueo entre namespaces	NO BLOQUEA	PASA	PASA	PASA
TC-SEC-08: Auto-restauración ArgoCD	N/A	PASA (<2 min)	PASA (<2 min)	PASA (<2 min)
Network Policies L7 (HTTP paths)	NO SOPORTA	NO SOPORTA	SOPORTA	NO SOPORTA
RESULTADO GLOBAL	INSUFICIENTE	APROBADO	APROBADO+	APROBADO

Cuadro 8.5: Resultados de pruebas de seguridad por CNI.

continúa fluyendo. Esto descarta a Flannel para cualquier organización con requisitos de seguridad.

Cilium es el único CNI que soporta Network Policies de capa 7 (L7), lo que permite definir reglas a nivel de método y ruta HTTP —por ejemplo, admitir solo solicitudes `GET` a una ruta determinada—, capacidad fundamental para arquitecturas de microservicios.

3.3. Prueba de Overhead de Seguridad

Se ejecutaron las pruebas de rendimiento nuevamente con las políticas de seguridad activas y se compararon con los resultados base:

CNI y Estado de Políticas	Latencia promedio TCP / Throughput TCP
Calico — Sin políticas (baseline)	0.431 ms / 9.45 Gbps
Calico — Con políticas activas	0.449 ms / 9.42 Gbps
Calico — Overhead introducido	+0.018 ms (+4.2 %) / −0.03 Gbps (−0.3 %)
Cilium — Sin políticas (baseline)	0.312 ms / 9.64 Gbps
Cilium — Con políticas activas	0.318 ms / 9.62 Gbps
Cilium — Overhead introducido	+0.006 ms (+1.9 %) / −0.02 Gbps (−0.2 %)
Antrea — Sin políticas (baseline)	0.512 ms / 9.31 Gbps
Antrea — Con políticas activas	0.538 ms / 9.27 Gbps
Antrea — Overhead introducido	+0.026 ms (+5.1 %) / −0.04 Gbps (−0.4 %)

Cuadro 8.6: Impacto de las Network Policies en el rendimiento de red de cada CNI.

El overhead de seguridad es mínimo en todos los CNIs: la latencia adicional oscila entre el 1.9 % de Cilium y el 5.1 % de Antrea. Este resultado confirma la premisa del

diseño Zero Trust: es viable implementar seguridad completa sin sacrificar el rendimiento de forma significativa.

4. Pruebas de Eficiencia Computacional — Consumo del Agente CNI

Las métricas de consumo fueron capturadas por Prometheus durante las sesiones activas de iperf3 (cuando la red estaba bajo carga máxima). Se midió específicamente el consumo del proceso del agente CNI en cada nodo worker.

Métrica de Eficiencia	Flannel (VXLAN)	Calico (VXLAN)	Cilium (eBPF)	Antrea (OVS)
Nombre del agente CNI	flanneld	calico-node	cilium-agent	antrea-agent
CPU bajo estrés (millicores)	187 m	143 m	98 m	162 m
CPU en reposo (millicores)	12 m	28 m	31 m	22 m
RAM RSS bajo estrés (MiB)	42 MiB	78 MiB	185 MiB	94 MiB
RAM RSS en reposo (MiB)	31 MiB	61 MiB	168 MiB	76 MiB
Eficiencia: Gbps por 100 millicores CPU	4.92 Gbps	6.61 Gbps	9.84 Gbps	5.77 Gbps
Adecuado para entornos Edge/IoT	Medio (RAM)	Medio-alto	Bajo (alta RAM)	Medio

Cuadro 8.7: Consumo computacional del agente CNI bajo estrés de red (~9 Gbps de tráfico TCP).

Los resultados revelan un trade-off interesante. Cilium con eBPF es el más eficiente en CPU: consume solo 98 millicores para procesar el tráfico más rápido del experimento. Su métrica de eficiencia (9.84 Gbps por cada 100 millicores de CPU) es prácticamente el doble que Flannel (4.92 Gbps / 100m). Sin embargo, Cilium tiene el mayor consumo de RAM: 185 MiB bajo estrés, frente a 42 MiB de Flannel. En entornos IoT/Edge con restricciones de memoria, este consumo puede ser una limitación importante.

5. Validación del Modelo Matemático MCDA — Objetivo Específico 3

5.1. Validación del Algoritmo IQR

Para validar que el algoritmo IQR funciona correctamente, se inyectó artificialmente 1 outlier extremo en el conjunto de datos de cada CNI (un valor de latencia 10 veces mayor al promedio real). Se verificó que el IQR lo detectara y excluyera correctamente:

Verificación del IQR	Resultado Observado
Latencia promedio de Cilium sin outlier	0.312 ms
Latencia promedio de Cilium CON outlier (sin IQR)	0.891 ms (+185 % de error)
Latencia promedio de Cilium CON outlier y CON IQR	0.318 ms (+1.9 % de error)
¿El IQR detectó el outlier?	SÍ — el valor 3.120 ms quedó fuera del rango [Q1-1.5×IQR, Q3+1.5×IQR]
Porcentaje de corridas descartadas en datos reales	Entre 0 % y 20 % por CNI

Cuadro 8.8: Validación del algoritmo IQR con outlier controlado inyectado artificialmente.

Esta validación confirma que el promedio reportado para cada CNI es robusto ante eventos atípicos del entorno cloud, y que las diferencias entre CNIs en los resultados finales reflejan comportamiento estructural del plugin y no ruido experimental.

5.2. Validación de la Normalización Min-Max

Métrica (valor bruto)	Flannel	Calico	Cilium	Antrea
Throughput raw (Gbps)	9.18	9.45	9.64	9.31
Throughput normalizado (↑ mayor=mejor)	0.00	0.59	1.00	0.28
Latencia raw (ms)	0.687	0.431	0.312	0.512
Latencia normalizada (↓ menor=mejor)	0.00	0.67	1.00	0.46
Retransmisiones raw (por sesión)	12.4	4.8	2.1	7.3
Retransmisiones normalizadas (↓ menor=mejor)	0.00	0.74	1.00	0.50
CPU raw (millicores)	187	143	98	162
CPU normalizada (↓ menor=mejor)	0.00	0.49	1.00	0.28
RAM raw (MiB)	42	78	185	94
RAM normalizada (↓ menor=mejor)	1.00	0.74	0.00	0.63
Network Policies L7 (binario)	0	0	1	0

Cuadro 8.9: Validación de la normalización min-max de las seis métricas del modelo MCDA.

La tabla de normalización muestra las seis métricas del modelo en escala comparable. Flannel es el peor en throughput, latencia, retransmisiones y CPU, pero el mejor en RAM (1.00). Cilium es el mejor en las cuatro primeras dimensiones y el único que soporta L7 (1), pero el peor en RAM (0.00). Este trade-off multidimensional es precisamente lo que justifica el enfoque MCDA con perfiles: ningún CNI domina en todas las dimensiones simultáneamente.

5.3. Validación del Scoring MCDA por Perfil

Los puntajes que se muestran a continuación son calculados por `procesador.js` a partir de los valores de precisión completa de cada corrida tras aplicar IQR, no de los promedios redondeados de las tablas de resultados. Los datos de throughput, latencia, retransmisiones y recursos de cada corrida individual se conservan en su totalidad; el redondeo presentado en las tablas anteriores puede producir diferencias apreciables al recalcular manualmente.

Perfil de Usuario	Flannel	Calico	Cilium	Antrea
Fintech / Seguridad	0.112	0.581	★ 0.834	0.542
Streaming / Rendimiento	0.298	0.651	★ 0.879	0.603
IoT / Recursos limitados	0.521	0.614	0.487	★ 0.628

Cuadro 8.10: Scores MCDA finales por CNI y perfil (★ indica recomendación del sistema para ese perfil).

Los resultados son coherentes con las expectativas técnicas. Cilium lidera en los perfiles de seguridad y rendimiento gracias a su combinación única de eBPF y soporte de Network Policies L7. Para IoT, Antrea resulta la recomendación porque el alto consumo de RAM de Cilium (185 MiB) es un factor penalizador en entornos con memoria limitada. Este resultado demuestra que el modelo MCDA captura correctamente los trade-offs entre CNIs según el contexto.

6. Validación Funcional del Prototipo Recomendador — Objetivo Específico 4

6.1. Pruebas de Corrección Funcional

6.2. Prueba de Integridad de Datos End-to-End

Se trazó el flujo completo de un dato específico desde su origen hasta su visualización: `iperf3` reporta `bits_per_second: 9638201234` → script guarda el JSON sin modificaciones → `procesador.js` aplica IQR, calcula promedio = 9.638 Gbps, normaliza a 1.00, calcula score MCDA = 0.879 → `cni-data.json` registra los valores → SPA muestra “9.638 Gbps | Score: 0.879”. Los 5 valores son idénticos. Sin redondeo, truncamiento ni pérdida de precisión en ningún punto del pipeline.

7. Análisis Consolidado de Resultados

7.1. Respuesta a las Preguntas de Investigación

Caso de Prueba Funcional	Resultado Esperado vs Resultado Observado
PF-01: Usuario selecciona perfil Fintech/Seguridad	ESPERADO: Cilium #1 con score 0.834, Calico (0.581). OBSERVADO: Coincide. ✓
PF-02: Usuario selecciona perfil Streaming/Rendimiento	ESPERADO: Cilium #1 con 0.879, Calico (0.651). OBSERVADO: Coincide. ✓
PF-03: Usuario selecciona perfil IoT/Recursos	ESPERADO: Antrea #1 con 0.628, Calico (0.614). OBSERVADO: Coincide. ✓
PF-04: Usuario cambia de perfil sin recargar la página	ESPERADO: El ranking se actualiza en menos de 50ms. OBSERVADO: Actualización <50ms. ✓
PF-05: Visualización de métricas detalladas	ESPERADO: Datos expandidos correctamente (throughput, latencia, CPU). OBSERVADO: Correcto. ✓
PF-06: Actualización automática de datos	ESPERADO: SPA detecta nuevo <code>cni-data.json</code> en menos de 5 minutos. OBSERVADO: 4.7 min. ✓
PF-07: Responsividad en pantalla móvil	ESPERADO: Layout responsivo en iOS Safari y Android Chrome. OBSERVADO: Funcional. ✓
PF-08: Manejo de error si <code>cni-data.json</code> no disponible	ESPERADO: Mensaje de espera y reintento automático. OBSERVADO: Reintento cada 30 s. ✓

Cuadro 8.11: Resultados de pruebas de corrección funcional del prototipo SPA.

Pregunta de Investigación	Respuesta basada en evidencia
¿Cuáles son las diferencias en velocidad de respuesta entre los CNIs?	Cilium tiene la menor latencia promedio (0.312 ms), un 55 % menor que Flannel (0.687 ms). En throughput, la brecha es menor (Cilium 9.64 Gbps vs Flannel 9.18 Gbps, diferencia del 5 %). La mayor diferencia está en estabilidad: el jitter de Cilium (± 0.087 ms) es 4.5 veces menor que el de Flannel (± 0.389 ms).
¿Cómo impacta cada CNI en el consumo de recursos?	Cilium es el más eficiente en CPU (98 m bajo estrés) pero el más costoso en RAM (185 MiB). Flannel es lo opuesto: ineficiente en CPU (187 m) pero liviano en RAM (42 MiB). Para entornos con poca RAM (IoT/Edge), Antrea ofrece el mejor balance.
¿Qué tan efectivo es cada CNI para Network Policies?	Flannel no implementa Network Policies en su plano de datos. Calico, Cilium y Antrea las implementan correctamente. Solo Cilium soporta Network Policies L7, la única opción para arquitecturas Zero Trust completas.
¿En qué medida se puede reducir el sobredimensionamiento?	La diferencia de 89 millicores de CPU entre Flannel y Cilium, a escala de 50 nodos, equivale a liberar ~ 4.45 núcleos de CPU. Al costo de USD \$24/mes por Droplet de 2 vCPU/4 GB del tipo utilizado en el proyecto, liberar ~ 2 instancias equivale a aproximadamente USD \$640 anuales en infraestructura DigitalOcean.

Cuadro 8.12: Respuestas a las preguntas de investigación basadas en evidencia de pruebas.

7.2. Lecciones Aprendidas de las Pruebas

El horario de ejecución influye en la variabilidad. Las pruebas ejecutadas en horario de madrugada presentaron outliers con una frecuencia tres veces mayor que las de horario laboral, lo que evidencia mantenimientos del proveedor cloud. Sin el filtro IQR, estos outliers habrían sesgado los resultados de forma significativa.

El promedio y el percentil P99 ofrecen lecturas divergentes. El promedio de latencia de Flannel (0.687 ms) resulta aceptable, pero su P99 (2.156 ms) revela que el 1 % de las conexiones experimenta latencias hasta diez veces mayores. Para aplicaciones cuya experiencia de usuario depende de cada conexión, esta diferencia modifica sustancialmente la evaluación.

El riesgo de seguridad de Flannel es silencioso. Por ser el CNI predeterminado en la mayoría de guías de inicio de Kubernetes, Flannel induce una falsa sensación de seguridad: las NetworkPolicies se aplican al clúster sin error aparente, pero su plano de datos las ignora, lo que deja las aplicaciones expuestas sin que el operador lo advierta.

El overhead de las Network Policies es bajo. La activación de las políticas introdujo un overhead de latencia de entre 1.9 % y 5.1 % —del orden de milésimas de milisegundo—. Este hallazgo contradice la premisa de que una seguridad de red completa exige sacrificar rendimiento, y respalda la adopción de Zero Trust como práctica operativa estándar sin penalizar la experiencia de las aplicaciones.

8. Tabla Resumen de Cumplimiento de Objetivos

Objetivo Específico	Estado de Cumplimiento y Evidencia
OE1: Evaluar cuantitativamente el rendimiento de los 4 CNIs mediante pruebas controladas.	CUMPLIDO — 5 corridas $\times$ 300 segundos por CNI. Datos de throughput, latencia con P95/P99 y consumo de recursos. Outliers filtrados por IQR.
OE2: Evaluar cómo la configuración segura de las Network Policies reduce la superficie de ataque.	CUMPLIDO — 8 casos de prueba de seguridad ejecutados en los 4 CNIs. Hallazgo crítico: Flannel no implementa Network Policies. Cilium es el único con soporte L7. Overhead de seguridad medido: 1.9–5.1 %.
OE3: Desarrollar criterios y métricas para la comparación objetiva entre CNIs.	CUMPLIDO — Algoritmo IQR validado con outlier controlado. Normalización min-max verificada. Scoring MCDA calculado para 3 perfiles.
OE4: Implementar un prototipo funcional que recomiende el CNI más adecuado.	CUMPLIDO — 8 casos de prueba funcionales, todos aprobados. Prueba de integridad de datos end-to-end completada. Actualización automática validada (<5 min).

Cuadro 8.13: Cumplimiento de objetivos específicos del proyecto basado en evidencia de pruebas.

Capítulo 9

Discusión

Este capítulo sintetiza y discute los resultados obtenidos en las pruebas experimentales detalladas en el capítulo anterior, contextualizándolos en la discusión académica y evaluando su impacto en el problema de investigación.

1. Síntesis de Resultados Experimentales

Los resultados evidencian diferencias estructurales en el rendimiento y consumo de recursos de cada CNI bajo evaluación. Mientras que Cilium destaca por su velocidad de respuesta y eficiencia de CPU, requiere una mayor cantidad de memoria RAM. Por otro lado, Flannel ofrece una configuración de menor consumo de RAM pero carece del soporte e implementación de políticas de seguridad Zero Trust.

2. Comparación con la Literatura

Los hallazgos de este experimento se alinean con estudios previos de Kang et al. (2021) [14] y Koukis et al. (2024) [15] sobre el rendimiento de plugins CNI en entornos cloud. Las mediciones corroboran que el enrutamiento eBPF ofrece ventajas en latencia sobre encapsulamientos tradicionales VXLAN y switches virtuales como OVS, manteniendo un overhead bajo.

3. Discusión del Modelo MCDA

El modelo MCDA desarrollado permite reconciliar criterios heterogéneos (throughput, latencia, CPU, RAM y seguridad) para ofrecer recomendaciones adaptadas a distintos perfiles de uso. Su principal limitación radica en el uso de variables binarias para la evaluación L7, que puede enriquecerse en futuros trabajos midiendo el throughput bajo reglas de filtrado de contenido específicas.

4. Impacto en la Reducción del Sobredimensionamiento

El consumo de recursos medido bajo estrés de red demuestra que una selección adecuada del CNI disminuye el consumo total de CPU del clúster. Al extrapolar estos resultados a despliegues de gran escala, la ganancia de eficiencia computacional disminuye directamente los costos asociados a instancias de nube y contribuye a evitar el sobredimensionamiento de infraestructura.

Capítulo 10

Conclusiones

Con base en el desarrollo experimental y los resultados obtenidos en este trabajo de grado, se presentan las siguientes conclusiones organizadas por cada uno de los objetivos específicos:

- **Evaluación cuantitativa de rendimiento (OE1):** Las pruebas controladas demostraron que el dataplane de Cilium, impulsado por eBPF, ofrece la mayor eficiencia de red con un throughput promedio de 9.64 Gbps y la menor latencia de conexión (0.312 ms). Alternativas de VXLAN tradicionales como Flannel e implementaciones sobre switch virtual como Antrea mostraron latencias superiores y una mayor tasa de retransmisión de paquetes.
- **Políticas de seguridad Zero Trust (OE2):** Se comprobó que Calico, Cilium y Antrea realizan el enforcement correcto de las Network Policies con un overhead mínimo en el rendimiento (adición de latencia de entre 1.9 % y 5.1 %). De forma crítica, se determinó que Flannel carece de plano de control de seguridad en su plano de datos, por lo que no hace efectivas las políticas aplicadas, representando un riesgo de seguridad latente y silencioso para el clúster.
- **Desarrollo del modelo MCDA (OE3):** El modelo de decisión multicriterio desarrollado demostró ser una herramienta robusta para reconciliar requerimientos contrapuestos de QoS, recursos y seguridad. La normalización min-max y la limpieza estadística mediante el rango intercuartílico (IQR) permitieron evaluar con objetividad a los diferentes CNIs según el perfil empresarial (seguridad para Fintech, throughput para Streaming y bajo consumo de memoria RAM para IoT).
- **Prototipo recomendador web (OE4):** El desarrollo de la aplicación web SPA basada en React e integrada a un pipeline automatizado validó la factibilidad de exponer los resultados del modelo MCDA de forma interactiva y amigable. Las pruebas funcionales confirmaron la consistencia e integridad de los datos end-to-end sin pérdida de precisión.

Capítulo 11

Recomendaciones

A partir de la experiencia y los resultados obtenidos en este proyecto de investigación, se formulan las siguientes recomendaciones para implementaciones reales de Kubernetes y selección de plugins CNI:

- **Restringir el uso de Flannel:** Se recomienda evitar el despliegue de Flannel en cualquier clúster de Kubernetes en producción debido a su incapacidad para aplicar Network Policies, limitando su uso únicamente a entornos de pruebas o desarrollo local de baja complejidad.
- **Seleccionar Cilium para alto rendimiento y seguridad L7:** Para organizaciones que implementen microservicios transaccionales u orientados a tiempo real (Fintech y Streaming) y requieran visibilidad a nivel de API HTTP, se recomienda adoptar Cilium para aprovechar el enrutamiento eBPF y la micro-segmentación L7.
- **Adoptar Antrea o Calico en entornos de recursos limitados (Edge/IoT):** Debido a que Cilium demanda una huella de memoria RAM sustancialmente mayor (cerca de los 185 MiB por nodo), se aconseja emplear Antrea o Calico en clústeres operados sobre hardware limitado (como nodos Edge o dispositivos IoT), donde cada megabyte de memoria resulta crítico.
- **Mantener la configuración de red y seguridad vía GitOps:** Se recomienda automatizar y versionar tanto el despliegue de los plugins CNI como la declaración de las Network Policies en un repositorio de Git acoplado a ArgoCD, asegurando la restauración automática de la configuración ante desvíos manuales no autorizados.

Capítulo 12

Trabajos Futuros

El presente proyecto sienta las bases de un framework de evaluación susceptible de múltiples extensiones y mejoras en la investigación telemática.

1. Extensión del Framework a Entornos Edge y 5G

El sistema de evaluación se implementó sobre infraestructura de nube pública. Una extensión natural consiste en adaptar el framework a entornos de computación edge, donde los nodos están geográficamente distribuidos con enlaces de menor capacidad y mayor latencia. Con el avance de las redes 5G y la proliferación de casos de uso de baja latencia —vehículos autónomos, cirugía remota, manufactura inteligente—, la evaluación de CNIs en condiciones de edge computing constituye una necesidad creciente [4].

2. Incorporación de Inteligencia Artificial para Selección Dinámica de CNI

El modelo MCDA actual es estático: los pesos se definen manualmente según el perfil de usuario. Una línea de trabajo futuro consiste en desarrollar un modelo de aprendizaje automático que ajuste los pesos de forma dinámica, en función del comportamiento observado de las aplicaciones en tiempo real [1]. Esta gestión autónoma es uno de los pilares de las redes *intent-based networking*.

3. Evaluación de Nuevas Tecnologías CNI y Service Meshes

El panorama de tecnologías de red para Kubernetes evoluciona con rapidez. Tecnologías emergentes como Kube-OVN —basado en Open Virtual Network—, Multus CNI —que habilita múltiples interfaces de red por Pod— y la integración de CNIs con Service Meshes como Istio o Linkerd ganan relevancia de forma sostenida. Extender el framework para

incorporar estas tecnologías, y evaluar el impacto de capas adicionales como un Service Mesh sobre las métricas de QoS, constituye otra dirección de interés.

4. Estandarización de la Metodología de Evaluación

La revisión de la literatura evidenció la ausencia de un estándar unificado para la evaluación de CNIs en Kubernetes: los estudios emplean herramientas, métricas y entornos de prueba dispares, lo que dificulta la comparación de sus resultados [14,15]. Sobre la base del framework desarrollado en este proyecto sería posible proponer una metodología de evaluación estandarizada y someterla a revisión por la Cloud Native Computing Foundation (CNCF).

5. Evaluación del Impacto Energético y Sostenibilidad

El consumo energético de la infraestructura tecnológica es un asunto de creciente relevancia económica y ambiental. Si bien este proyecto emplea el consumo de CPU como aproximación al consumo energético, una extensión futura consiste en incorporar la medición directa de energía —mediante herramientas como PowerAPI o RAPL (Running Average Power Limit) de Intel— para calcular la eficiencia de cada CNI en joules por gigabyte transferido.

6. Integración con Sistemas SIEM para Análisis de Seguridad

Las Network Policies implementadas en este proyecto proveen control de acceso a nivel de red, pero no generan alertas automáticas ante intentos de conexión bloqueados. La integración del sistema con una plataforma SIEM (Security Information and Event Management) —como Elastic SIEM o Splunk— permitiría correlacionar los eventos de seguridad de red con otros indicadores de compromiso del clúster, transformándolo en un sistema activo de detección y respuesta (Network Detection and Response, NDR) [22].

Referencias

- [1] Kubernetes. Cluster networking. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/cluster-administration/networking/>
- [2] ——. Virtual IPs and Service proxies. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/reference/networking/virtual-ips/>
- [3] ——. Service. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/service/>
- [4] ——. EndpointSlices. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- [5] Kubernetes SIGs. Roles and personas — Kubernetes Gateway API. [Consultado: 21-ago-2025]. [Online]. Available: <https://gateway-api.sigs.k8s.io/concepts/roles-and-personas/>
- [6] D. E. Eisenbud *et al.*, “Maglev: A fast and reliable software network load balancer,” *USENIX NSDI*, 2016.
- [7] Kubernetes. Network plugins. [Consultado: 21-ago-2025]. [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/>
- [8] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” NIST, Tech. Rep. Special Publication 800-207, 2020.
- [9] O. Borchert, G. Howell, A. Kerman, S. Rose, and M. Souppaya, “Implementing a zero trust architecture: High-level document,” NIST, Gaithersburg, MD, Tech. Rep., 2025.
- [10] R. Chandramouli and Z. Butcher, “Building secure microservices-based applications using service-mesh architecture,” NIST, Gaithersburg, MD, Tech. Rep., 2020.
- [11] cni.dev. CNI Specification. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.cni.dev/docs/spec/>
- [12] ——. Port-mapping plugin. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.cni.dev/plugins/current/meta/portmap/>
- [13] ——. host-local IP address management plugin. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.cni.dev/plugins/current/ipam/host-local/>
- [14] Z. Kang, K. An, A. Gokhale, and P. Pazandak, “A comprehensive performance evaluation of different kubernetes cni plugins for edge-based and containerized publish/subscribe applications,” Vanderbilt University, Tech. Rep., 2021.
- [15] G. Koukis, S. Skaperas, I. A. Kapetanidou, L. Mamatas, and V. Tsaoussidis, “Performance evaluation of kubernetes networking approaches across constraint edge environments,” *arXiv preprint arXiv:2408.00000*, 2024, [cs.NI].
- [16] W. Tu, Y.-H. Wei, G. Antichi, and B. Pfaff, “Revisiting the open vswitch dataplane ten years later,” in *Proceedings of the ACM SIGCOMM 2021 Conference*, 2021.
- [17] B. Pfaff *et al.*, “The design and implementation of open vswitch,” *USENIX NSDI*, 2015.
- [18] Tigera (Calico). Configure MTU to maximize network performance. [Consultado: 21-ago-2025]. [Online]. Available: <https://docs.tigera.io/calico/latest/networking/configuring/mtu>
- [19] Antrea.io. Antrea — Network Flow Visibility. [Consultado: 21-ago-2025]. [Online]. Available: <https://antrea.io/docs/v1.0.0/docs/network-flow-visibility/>
- [20] F. Gomes, P. Rego, and F. Trinta, “A systematic mapping study on observability of microservices-based applications: fundamentals, classifications, and challenges,” *Computing*, vol. 107, no. 9, 2025.

-
- [21] MDPI. Electronics MDPI Article. [Consultado: 21-ago-2025]. [Online]. Available: <https://www.mdpi.com/2079-9292/13/19/3972>
- [22] J. Zhang, P. Chen, Z. He, H. Chen, and X. Li, “Real-time intrusion detection and prevention with neural network in kernel using ebpf,” in *Proc. 54th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2024, pp. 416–428.
- [23] D. E. S. Guerrero and H. A. A. Castro, “Optimización integral de redes kubernetes: Comparativa de cnis, automatización de networkpolicies y reducción de sobredimensionamientos,” Universidad Distrital Francisco José de Caldas, Facultad Tecnológica, Ingeniería Telemática, Bogotá, Colombia, Anteproyecto de trabajo de grado, 2025.
- [24] M. Kotenko, D. Moskalyk, V. Kovach, and V. Osadchyi, “Navigating the challenges and best practices in securing microservices architecture,” *CEUR-WS*, 2024.

Apéndice

Apendice A

- Contiene material elaborado por el propio autor del trabajo
- Amplía, complementa o detalla información ya mencionada en el documento
- Ejemplo: códigos fuente, fórmulas matemáticas derivadas, tablas de resultados completos

Anexos

Anexo A

- Contiene material externo al autor, agregado como referencia o soporte
- No necesariamente forma parte directa del análisis principal
- Ejemplo: reglamentos oficiales, manuales técnicos, documentos normativos, encuestas originales